

Introdução à linguagem Java

Bib: Java: An Introduction to Problem Solving & Programming, Walter Savitch, cap. 1 a 4

Programação III (PG3)

João Pedro Patriarca, (joao.patriarca@isel.pt),

Gabinete F.0.23 do edifício F, LEETC, ADEETC

Slides inspirados nos slides da professora Manuela Sousa



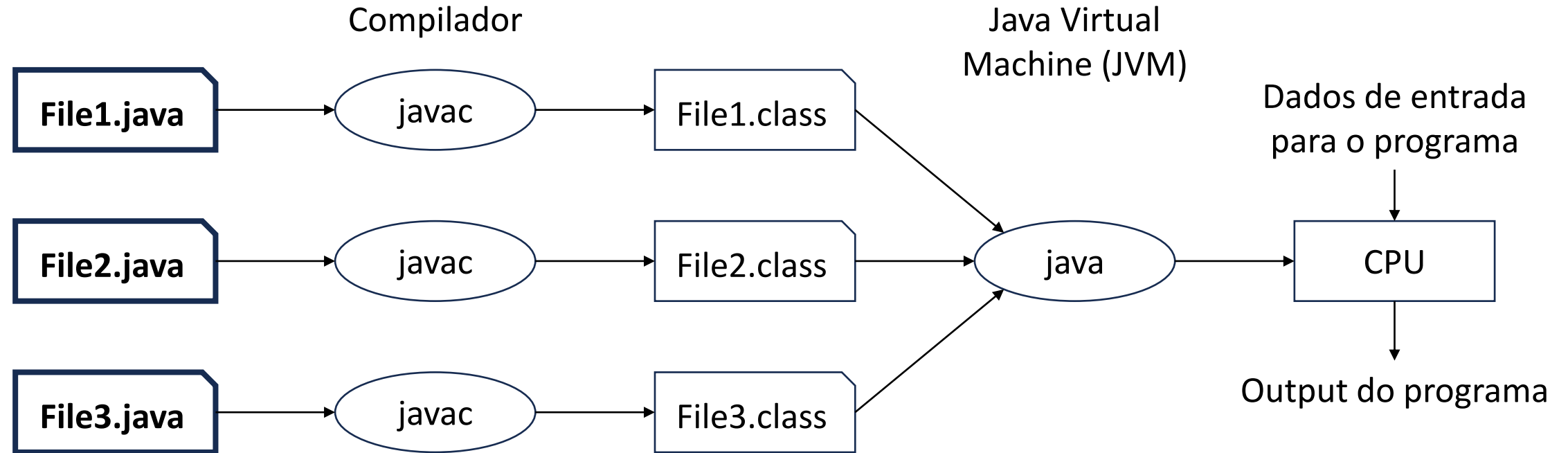
Resumo

- Visão geral do Java (cp. 1)
- Tipos primitivos (cp. 2)
- *Strings* (cp. 2)
- Escrita e leitura da consola (cp. 2)
- Controlo de fluxo (cp. 3 e 4)

Resumo

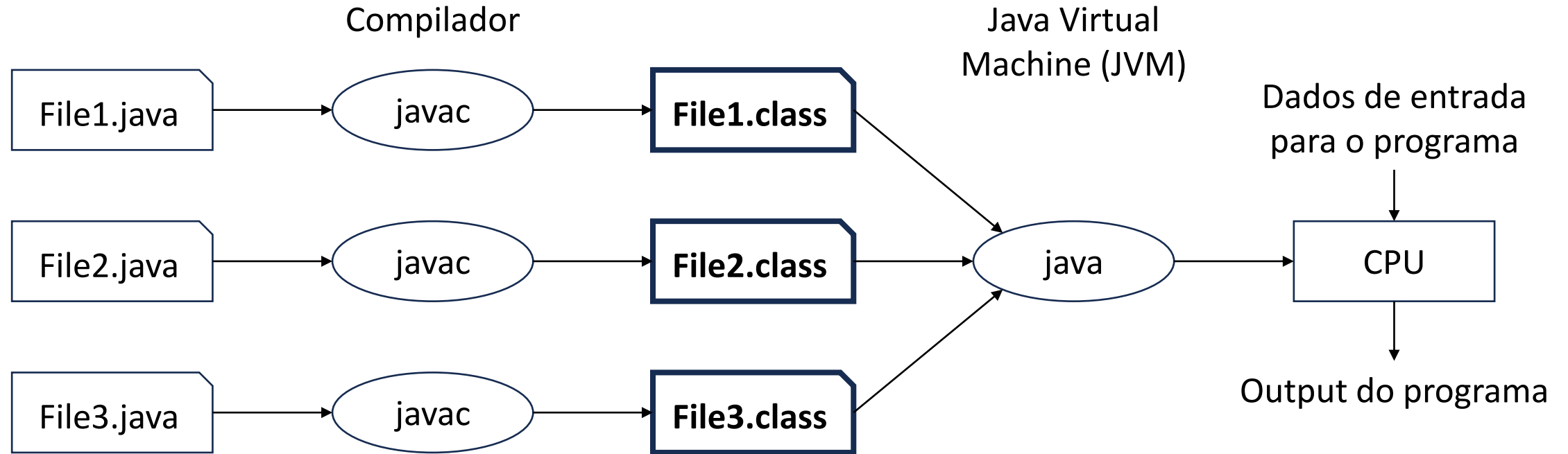
- Visão geral do Java (cp. 1)
- Tipos primitivos (cp. 2)
- *Strings* (cp. 2)
- Escrita e leitura da consola (cp. 2)
- Controlo de fluxo (cp. 3 e 4)

Produção e execução de um programa Java (1 de 3)



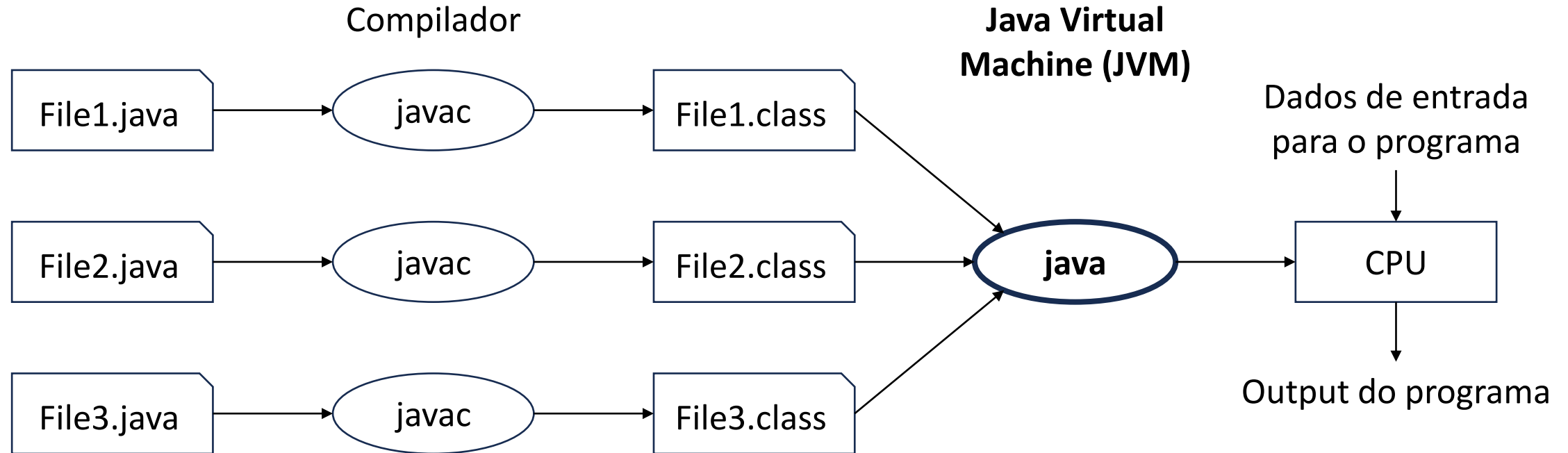
- Um programa é constituído, por norma, por vários ficheiros
- Cada ficheiro com código fonte tem extensão *.java*
- Cada ficheiro inclui a definição de uma única classe com visibilidade pública
- O nome da classe e o nome do ficheiro têm de ser iguais
- Cada ficheiro é compilado separadamente

Produção e execução de um programa Java (2 de 3)



- O ficheiro compilado tem extensão `.class`
- Contém byte-codes Java
 - Linguagem intermédia, transversal a qualquer arquitetura/plataforma
 - O compilador gera sempre o mesmo código IL, independentemente da arquitetura

Produção e execução de um programa Java (3 de 3)



- A JVM é específica para arquitetura e plataforma alvo
- Inclui um interpretador que converte o código IL (*byte-codes Java*) em código máquina para ser executado pelo CPU
 - Código IL é convertido uma única vez mesmo que executado múltiplas vezes e em diferentes momentos
- Inclui um Class Loader responsável por ligar e carregar uma classe
 - Operação realizada apenas quando a classe é necessária e funciona como um *linker* noutras linguagens

Java é uma linguagem OOP (*Object-Oriented Programming*)

- O que significa OOP?

Java é uma linguagem OOP (*Object-Oriented Programming*)

- O que significa OOP?
 - Conjunto de objetos que interagem entre si através de ações
 - De uma forma geral, incluem estado
 - A execução de uma ação promove a alteração de estado do objeto

Java é uma linguagem OOP (*Object-Oriented Programming*)

- O que significa OOP?
 - Conjunto de objetos que interagem entre si através de ações
 - De uma forma geral, incluem estado
 - A execução de uma ação promove a alteração de estado do objeto
- Ex: Um programa que faz a gestão de um parque de estacionamento. Há um objeto que é o parque que tem a noção do número de lugares disponíveis e um conjunto de objetos que representam carros que entram e saem do parque de estacionamento

Java é uma linguagem OOP (*Object-Oriented Programming*)

- O que significa OOP?
 - Conjunto de objetos que interagem entre si através de ações
 - De uma forma geral, incluem estado
 - A execução de uma ação promove a alteração de estado do objeto
- Ex: Um programa que faz a gestão de um parque de estacionamento. Há um objeto que é o parque que tem a noção do número de lugares disponíveis e um conjunto de objetos que representam carros que entram e saem do parque de estacionamento
- Objetos do mesmo tipo são instâncias de uma classe
- Campos de uma classe: caracterizam o estado de um objeto; objetos do mesmo tipo podem ter estados diferentes
- Métodos de uma classe: funções com código que modelam o comportamento de um objeto; objetos do mesmo tipo têm os mesmos métodos

Princípios de uma linguagem OOP

- Encapsulamento
- Herança
- Polimorfismo

Princípios de uma linguagem OOP

- Encapsulamento
 - Esconde os detalhes de implementação expondo apenas o necessário e suficiente para poder ser usado
- Herança
- Polimorfismo

Princípios de uma linguagem OOP

- Encapsulamento
 - Esconde os detalhes de implementação expondo apenas o necessário e suficiente para poder ser usado
- Herança
 - Conjunto de classes que implementam uma relação de herança, ou seja, herdam estado e comportamento
- Polimorfismo

Princípios de uma linguagem OOP

- Encapsulamento
 - Esconde os detalhes de implementação expondo apenas o necessário e suficiente para poder ser usado
- Herança
 - Conjunto de classes que implementam uma relação de herança, ou seja, herdam estado e comportamento
- Polimorfismo
 - Polimorfismo significa “várias formas”
 - A chamada do mesmo método pode realizar ações diferentes, função do tipo do objeto

Princípios de uma linguagem OOP

- Encapsulamento
 - Esconde os detalhes de implementação expondo apenas o necessário e suficiente para poder ser usado
- Herança
 - Conjunto de classes que implementam uma relação de herança, ou seja, herdam estado e comportamento
- Polimorfismo
 - Polimorfismo significa “várias formas”
 - A chamada do mesmo método pode realizar ações diferentes, função do tipo do objeto

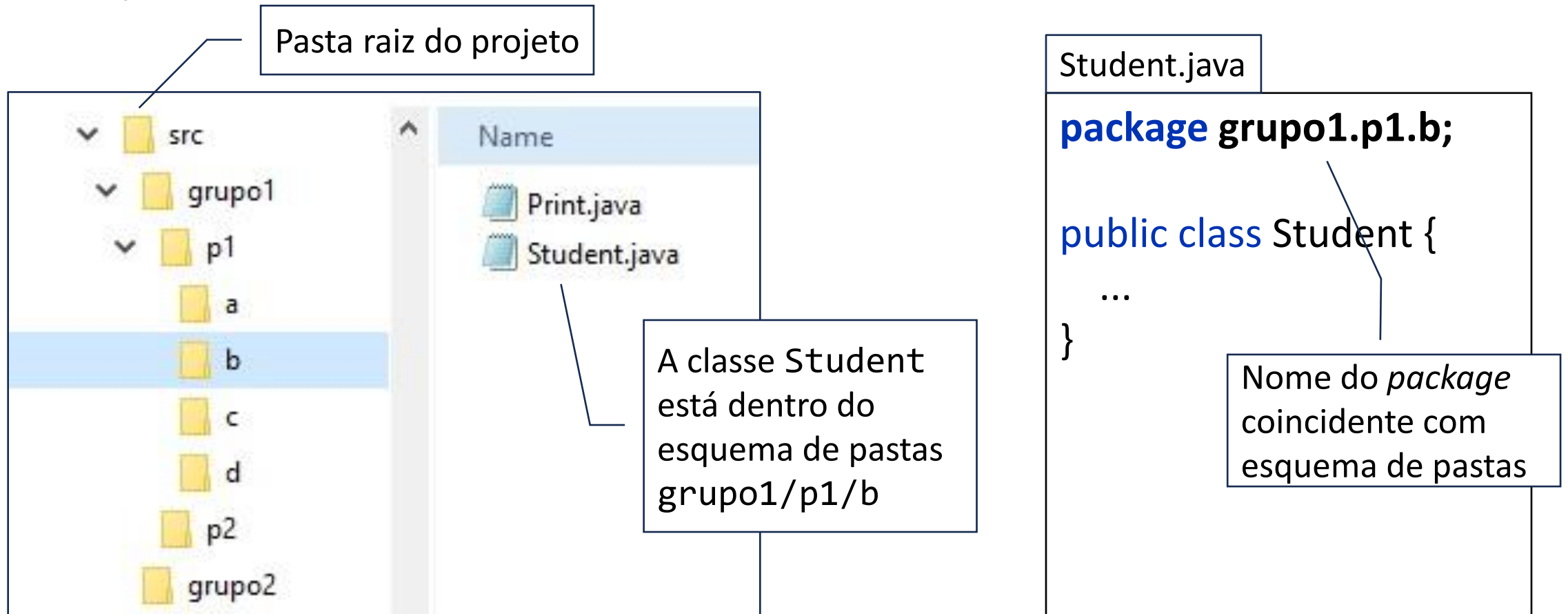
Temas aprofundados no futuro

Organização do código em Java

- Pacotes
 - Inclui classes e outros pacotes
 - Classes dentro de um pacote são classes relacionadas (ex: podem corresponder ao conjunto de classes que formam uma biblioteca)
- Classes
 - Define um tipo complexo/composto
 - Definição de uma classe por ficheiro e nome da classe igual ao nome do ficheiro com extensão *.java*
 - Contém métodos e campos
- Campos
 - Caracterizam estado (dados)
 - Podem corresponder a tipos primitivos ou outras classes
- Métodos
 - Modelam comportamento
 - Código com a sequência de instruções que vão ser executadas quando o método for chamado
 - Podem ser parametrizados e podem retornar um valor

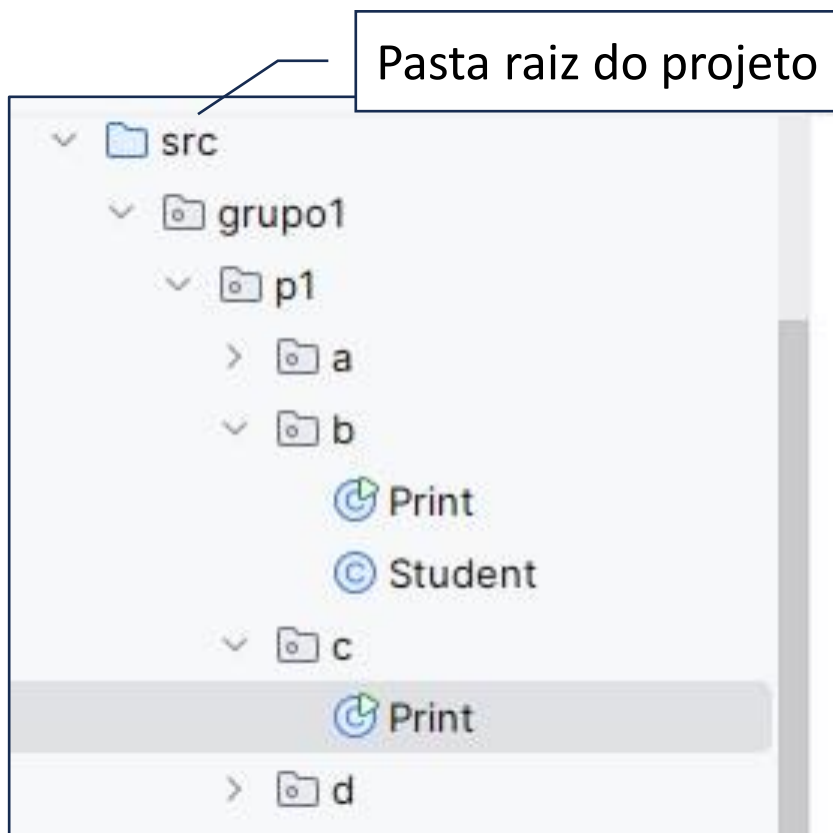
Pacotes (*packages*)

- Existe uma relação direta entre o nome de um *package* e a hierarquia de pastas no sistema de ficheiros



Importação de um *package* (**import**)

- Uma classe que use uma classe definida em outro *package* deve importar esse *package* (**import**)
- Fornece ao compilador o *pathname* da classe que está a ser usada



Fazer **import grupo1.p1.b.*;** importa todas as classes do *package*

```
package grupo1.p1.c;  
import grupo1.p1.b.Student;
```

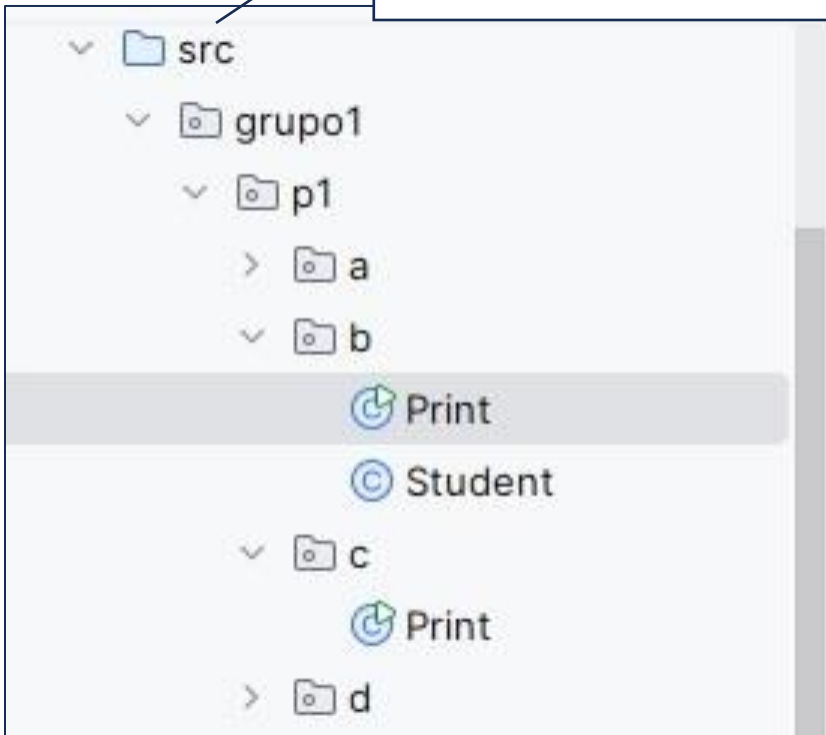
A classe Print usa a classe Student

```
public class Print {  
    public static void main(String[] args) {  
        Student s1 = new Student(3, "cc", 10);  
        Student s2 = new Student(3, "cc", 10);  
        ...  
    }  
}
```

Importação de um *package* (**import**)

- Classes do mesmo *package* não precisam fazer **import**

Pasta raiz do projeto



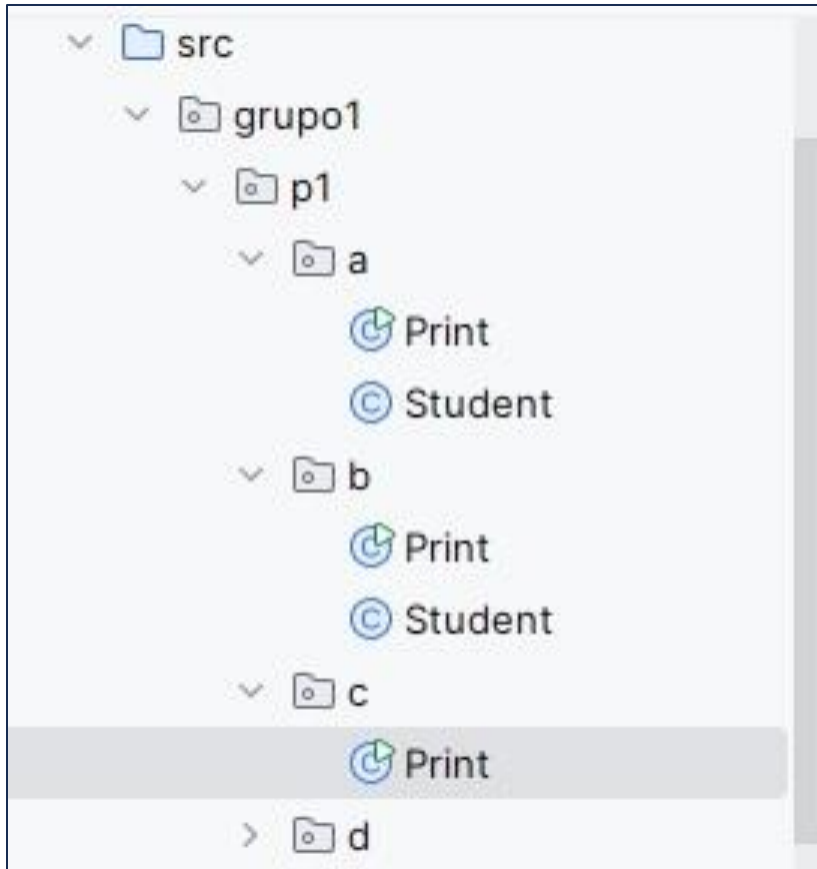
```
package grupo1.p1.b;
```

```
public class Print {  
    public static void main(String[] args) {  
        Student s = new Student(3, "bb", 10);  
        ...  
    }  
}
```

A classe Print e a classe Student pertencem ao mesmo *package*

Importação de um *package* (**import**)

- Utilização de duas classes com o mesmo nome em diferentes *packages*



```
package grupo1.p1.c;
```

```
public class Print {  
    public static void main(String[] args) {  
        grupo1.p1.a.Student s1 =  
            new grupo1.p1.a.Student(4, "cc1", 15);  
        grupo1.p1.b.Student s2 =  
            new grupo1.p1.b.Student(4, "cc2", 17);  
        ...  
    }  
}
```

A classe Print usa dois objetos Student diferentes: um Student do *package* grupo1.p1.a e outro Student do *package* grupo1.p1.b

Compilação de um programa na linha de comando

```
src> dir grupo1\p1\b
    Student.java
src> dir grupo1\p1\c
    Print.java
src> javac grupo1/p1/c/Print.java
src> dir grupo1\p1\b
    Student.class
    Student.java
src> dir grupo1\p1\c
    Print.class
    Print.java
```

- Como a classe `Print` depende de `Student`, ao compilar *Print.java*, compila *Student.java* se não existir um *Student.class* atualizado

Execução de um programa na linha de comando

- Uma classe para ser executada tem de definir o método `main`

```
package grupo1.p1.c;  
import grupo1.p1.b.Student;  
public class Print {  
    public static void main(String[] args) {  
        Student s1 = new Student(3, "cc", 10);  
        ...  
    } }  
}
```

```
src> java grupo1.p1.c.Print  
...
```

Importação de classes externas à diretoria base

- O compilador precisa ter informação onde procurar as classes no sistema de ficheiros
- Por omissão, procura com base na diretoria corrente e com base em algumas diretorias pré-definidas

Importação de classes externas à diretoria base

```
/*package grupo1.p1.c;*/  
import grupo1.p1.b.Student;  
  
public class Print {  
    public static void main(String[] args) {  
        Student s1 = new Student(3, "cc", 10);  
    ...  
}
```

O compilador não encontra a hierarquia de pastas grupo1\p1\b a partir da diretoria base corrente C:\...\p1\c

```
C:\CodeGarden\Trab1\src\grupo1\p1\c>javac Print.java  
Print.java:3: error: package grupo1.p1.b does not exist  
import grupo1.p1.b.Student;  
                ^  
...  

```

Importação de classes externas à diretoria raiz

- Para acrescentar outras diretorias na pesquisa é preciso adicionar essas diretorias na variável de ambiente *CLASSPATH*

%CLASSPATH% mantém as variáveis que já existem

O carácter ';' é o separador de diretorias

O carácter '.' representa a diretoria corrente

```
C:\CodeGarden\Trab1\src\grupo1\p1\c>set
CLASSPATH=%CLASSPATH%;c:\CodeGarden\Trab1\src\;
C:\CodeGarden\Trab1\src\grupo1\p1\c>javac Print.java

C:\CodeGarden\Trab1\src\grupo1\p1\c>java Print
...
```

Classificação de erros

- Erro de sintaxe
- Erro de execução
- Erro lógico

Classificação de erros

- Erro de sintaxe
 - A escrita de um programa obedece a regras (sintaxe)
 - Regras não respeitadas correspondem a erros gramaticais identificados pelo compilador
 - A classe não é compilada (não é gerado o ficheiro *.class*)
- Erro de execução
- Erro lógico

Classificação de erros

- Erro de sintaxe
 - A escrita de um programa obedece a regras (sintaxe)
 - Regras não respeitadas correspondem a erros gramaticais identificados pelo compilador
 - A classe não é compilada (não é gerado o ficheiro *.class*)
- Erro de execução
 - Erro gerado com o programa em execução
 - É gerada uma exceção que permite perceber e identificar a origem do erro
- Erro lógico

Classificação de erros

- Erro de sintaxe
 - A escrita de um programa obedece a regras (sintaxe)
 - Regras não respeitadas correspondem a erros gramaticais identificados pelo compilador
 - A classe não é compilada (não é gerado o ficheiro *.class*)
- Erro de execução
 - Erro gerado com o programa em execução
 - É gerada uma exceção que permite perceber e identificar a origem do erro
- Erro lógico
 - O programa compila com sucesso, executa sem gerar qualquer erro mas não produz o resultado correto
 - Erro de algoritmo

Classificação de erros

- Erro de sintaxe
 - A escrita de um programa obedece a regras (sintaxe)
 - Regras não respeitadas correspondem a erros gramaticais identificados pelo compilador
 - A classe não é compilada (não é gerado o ficheiro *.class*)
- Erro de execução
 - Erro gerado com o programa em execução
 - Em Java é gerada uma exceção que permite perceber e identificar a origem do erro
- Erro lógico
 - O programa compila com sucesso, executa sem gerar qualquer erro mas não produz o resultado correto
 - Erro de algoritmo

ATENÇÃO: a execução de um programa sem erros (de execução ou lógicos) não significa que esteja correto. Daí a relevância dos testes

Comentários

- Comentário de linha
 - Definido com dois caracteres *forward slash* (//)
 - Todo o texto desde os caracteres // até à mudança de linha é comentário
 - Ex: // *Isto é um comentário de linha*
- Comentário de bloco
 - O par de caracteres /* abre um comentário de bloco
 - O par de caracteres */ fecha um comentário de bloco
 - Pode se estender por uma ou mais linhas de texto
 - Ex: /* *Isto é um comentário de bloco* */
- Comentário JavaDoc
 - Semelhante ao comentário de bloco com a diferença que a abertura do comentário é concretizada com a sequência de caracteres /** (*forward slash* e dois asteriscos)
 - Usado para documentar classes, campos e métodos com a ferramenta *javadoc*
 - Ex: /** *Isto é um comentário javadoc* */

Resumo

- Visão geral do Java (cp. 1)
- **Tipos primitivos (cp. 2)**
- *Strings* (cp. 2)
- Escrita e leitura da consola (cp. 2)
- Controlo de fluxo (cp. 3 e 4)

Variáveis e valores

- Variáveis armazenam dados
 - Têm de ser declaradas antes de usadas
 - Têm de ser iniciadas antes de usadas
- Declaração/definição de uma variável:
<type> <identificador> [= <value>];
 - A declaração de uma variável não atribui um valor inicial
 - A definição de uma variável envolve a atribuição de um valor
 - *type* pode ser um tipo primitivo ou um tipo composto
 - O operador afetação (=), atribui um novo valor à variável
 - Os dados armazenados nas variáveis denominam-se *valores*

Identificadores

- O identificador é o nome de uma variável, de um tipo composto ou de um método
- Pode conter letras, dígitos e o caractere travessão ('_')
 - O nome de um identificador não pode começar por dígito
 - É *case sensitive*, ou seja, distingue identificadores cuja diferença está na utilização de letras maiúsculas ou minúsculas, no entanto, é má prática usar nomes iguais distintos apenas por este fator
- *Keywords* são palavras reservadas e, como tal, não podem ser usadas como identificadores
- Convenção de nomes:
 - Tipos classe começam com letra maiúscula
 - Variáveis, campos e métodos de uma classe começam com letra minúscula
 - Nomes com múltiplas palavras, cada palavra separa-se da palavra anterior com a primeira letra maiúscula (e.g. *umNomeMuitoComprido*)

Tipos em Java

- Tipos primitivos: valores simples que não permitem decomposição
 - Tipos inteiros: `byte`, `short`, `int` e `long`
 - Tipos com vírgula flutuante: `float` e `double`
 - Tipo caractere: `char`
 - Tipo booleano: `boolean`
- Tipos classe: objetos compostos que contêm dados e sobre os quais é possível realizar ações (métodos)
- Classes record: tipos especiais de classes que simplificam a modelação de dados agregados
- Tipos enumerados: tipo especial de dados que define um conjunto pré-definido de constantes

Tipos primitivos – domínios

Nome do tipo	Tipo de valor	Dimensão	Intervalo de valores
byte	inteiro	1 byte	-128 .. +127
short	inteiro	2 bytes	-32768 .. +32767
int	inteiro	4 bytes	-2147483648 .. 2147483647
long	inteiro	8 bytes	-9223372036854775808 .. +9223372036854775807
float	número com vírgula flutuante	4 bytes	$\pm 3.40282347 \times 10^{+38}$.. $\pm 1.40239846 \times 10^{-45}$
double	número com vírgula flutuante	8 bytes	$\pm 1.76769313486231570 \times 10^{+308}$.. $\pm 4.94065645841246544 \times 10^{-324}$
char	código de caractere Unicode	2 bytes	Todos os caracteres Unicode
boolean	true ou false	1 bit	Não aplicável

- Os domínios apresentados permanecem sempre os mesmos independentemente da arquitetura e do sistema

Literais ou constantes

- Um literal é um valor constante de um determinado tipo
 - Inteiro
 - Pode ser expresso na base decimal (por omissão), na base hexadecimal (prefixo 0x), na base octal (prefixo 0) e na base binária (prefixo 0b)
 - Pode ser precedido pelo sinal + ou -
 - O sufixo L expressa um literal do tipo long (e.g. 123L)
 - Vírgula flutuante
 - Expressos com dígitos depois do ponto (e.g. 1234560.0, -0.0000123) ou em notação científica (e.g. 1.23456e6, -1.23e-5)
 - O sufixo F expressa um literal do tipo float (e.g. 123.0F); sem o sufixo F, o literal é do tipo double
 - Caractere
 - Delimitado pelos caracteres plicas (e.g. 'a', 'Z')
 - *String*
 - Sequência de caracteres delimitada pelos caracteres aspas (e.g. "Java é fixe")

Definição de constantes

- Equivalente à definição de uma variável com um valor inicial mas imutável (constante)
 - O compilador gera erro se no programa existirem tentativas em alterar o valor
 - Por convenção o nome das constantes é sempre em maiúsculas
 - Inválido declarar a constante e afetá-la posteriormente
- Ex:
`public static final double PI = 3.14159;`
 - O que torna PI constante é o modificador `final`

Expressões e instruções

- Uma expressão produz um valor
- Uma expressão pode ser constituída por uma variável, literal, ou por uma combinação entre variáveis e literais usando operadores
- O tipo de operandos da expressão determinam o domínio em que as operações de uma expressão são executadas
- O resultado de uma expressão pode ser armazenado numa variável usando o operador afetação (=)
- Uma instrução é uma expressão terminada pelo caractere ‘;’

Conversão entre tipos primitivos

- Java é uma linguagem *fortemente tipificada (strongly typed)*
 - O compilador gera erro se a conversão não for implícita (e.g . a conversão implícita de um valor do tipo `double` para um inteiro gera erro de compilação)
- Conversão implícita
 - Quando a conversão for de um subdomínio para um superdomínio
byte → short → int → long → float → double
char → int → ...

Ex: `char a = 'z'; double b = a;`
- Conversão explícita (*type casting*)
 - O compilador exige que a conversão de um superdomínio para um subdomínio seja com conversão explícita
 - Pode implicar truncar o valor ou perda de precisão

Ex: `float c = 10.7F; byte d = (byte)c;`

Operadores aritméticos

- Operadores disponíveis:
 - adição (símbolo +), subtração (símbolo -), multiplicação (símbolo *) e divisão (símbolo /)
- Operadores binários, portanto envolvem dois operandos
- O tipo de dados do resultado é igual ao tipo de dados dos operandos
- Se os operandos tiverem tipos de dados diferentes, a operação é realizada no tipo de dados mais abrangente (variável **a** do tipo `int` e **b** do tipo `float`)
a+b produz um resultado do tipo `float`
- Numa expressão com várias operações, o cálculo da expressão é realizado passo a passo de acordo com a prioridade dos operadores
- A ordem das operações pode ser alterada com o uso de parênteses curvos
- As expressões em baixo produzem resultados diferentes (produzem resultados iguais para alguns valores particulares das variáveis)
a + b - c * d e **(a + b - c) * d**

Operador aritmético / e %

- No operador divisão, se os operandos forem do tipo inteiro, o resultado é igualmente do tipo inteiro
 - O resultado é truncado e não arredondado
 - Ex: seja a variável **a** do tipo `int` com o valor 3 e a variável **b** do tipo `int` com o valor 2, a expressão **a/b** produz o resultado 1
- No operador divisão, se os operandos forem do tipo real, o resultado é igualmente do tipo real
 - Ex: seja a variável **a** do tipo `int` com o valor 3 e a variável **b** do tipo `double` com o valor 2, a expressão **a/b** produz o resultado 1.5
- O operador *resto da divisão inteira* (%) deve ser usado com operandos do tipo inteiro
 - Ex: seja a variável **a** do tipo `int` com o valor 5 e a variável **b** do tipo `int` com o valor 3, a expressão **a%b** produz o resultado 2

Operador incremento e decremento em expressões

- O operador incremento (++) incrementa de 1 o valor da variável
- O operador decremento (--) decrementa de 1 o valor da variável
- Ambos podem ser usados como prefixo ou como pós-fixado da variável
- Ex:

```
int a = 10;
```

```
int b = (a++) + 5; // a = 11 e b = 15
```

```
int c = (++a) + 5; // a = 12 e c = 17
```

Expressões *booleanas*

- Uma expressão *booleana* produz sempre o valor `true` ou `false`, do tipo `boolean`
 - Ex: `boolean c = a < b;`
- Expressões *booleanas* podem ser combinadas usando os operadores `&&` (*And booleano*) e `||` (*Or booleano*)
 - Ex: `boolean c = a > 0 && a < b || a == 10`
- Uma expressão booleana pode ser negada com o operador `!` (*Not booleano*)
 - Ex: `boolean r = !(j > 0)`

Operadores de comparação

Notação matemática	Nome	Notação Java	Exemplo
=	Igual a	==	<code>balance == 0</code> <code>answer == 'y'</code>
≠	Diferente de	!=	<code>income != tax</code> <code>answer != 'y'</code>
>	Maior que	>	<code>expenses > income</code>
≥	Maior ou igual que	>=	<code>points >= 60</code>
<	Menor que	<	<code>pressure < max</code>
≤	Menor ou igual que	<=	<code>expenses <= income</code>

Avaliação curta (*short-circuit*)

- Uma expressão *booleana* pode não ser avaliada completamente
 - Numa expressão em *AND*, o primeiro termo que produza `false` determina o resultado final da expressão como `false`
 - Ex: `a >= 0 && a < b && c != 0`
 - Numa expressão em *OR*, o primeiro termo que produza `true` determina o resultado final da expressão como `true`
 - Ex: `a >= 0 || a < b || c != 0`
- Este aspeto, além de corresponder a uma otimização, pode ser imprescindível
 - Ex: `divisor != 0 && dividend / divisor > 5`
- A expressão *booleana* é avaliada completamente substituindo o operador `&&` pelo operador `&` e o operador `||` pelo operador `|`

Regras de precedência

- Numa expressão, as operações são realizadas pela ordem de precedência
- Os parênteses curvos podem alterar a ordem de avaliação das operações
 - Por vezes, a utilização de parênteses melhora a legibilidade da expressão, ainda que não altere a ordem de avaliação
- A avaliação é realizada da esquerda para a direita em operações binárias com a mesma precedência
- A avaliação é realizada da direita para a esquerda em operações unárias com a mesma precedência

Precedência dos operadores

Precedência mais alta	
Operador ponto, indexação e chamada a função	., [], ()
Operadores unários e conversão explícita	+, -, ++, --, ~, !, (Type)
Operadores multiplicativos	*, /, %
Operadores aditivos	+, -
Operadores de deslocamento	>>, <<, >>>
Operadores de comparação de superioridade	<, >, <=, >=, isinstance
Operadores de comparação de igualdade	==, !=
Operador AND bit a bit	&
Operador EOR bit a bit	^
Operador OR bit a bit	
Operador booleano AND	&&
Operador booleano OR	
Operador ternário	? :
Operadores de afetação	=, *=, /=, %=, +=, -=, <<=, >>=, >>>=, &=, ^=, =
Precedência mais baixa	

Operador ==

- O operador == é adequado para testar a igualdade entre dois números inteiros
- O operador == não é adequado para testar a igualdade entre dois números reais (`float` e `double`)
 - Para estes tipos deve usar-se os operadores de relação de superioridade (`>` e `<`)
 - Para verificar a igualdade, deve verificar-se se a diferença entre os dois valores é inferior a uma tolerância apropriada
 - Ex: `abs(a - b) < 0.1`
- O operador == não é adequado para testar a igualdade de dois objetos
 - Por exemplo, testar se duas *strings* são iguais com o operador ==, testa se correspondem ao mesmo objeto (se têm a mesma referência) e não se correspondem à mesma sequência de caracteres

Resumo

- Visão geral do Java (cp. 1)
- Tipos primitivos (cp. 2)
- *Strings* (cp. 2)
- Escrita e leitura da consola (cp. 2)
- Controlo de fluxo (cp. 3 e 4)

Classes versus tipos primitivos

- Uma classe serve para instanciar objetos
- Um objeto armazena dados e realiza ações através de métodos
- Objetos da mesma classe armazenam dados do mesmo tipo mas com valores diferentes e têm os mesmos métodos
- Tipos primitivos têm valores mas não têm métodos
- Uma *string* corresponde a uma sequência de caracteres terminados por 0 e, em Java, corresponde ao tipo `String` (tipo *built in* da linguagem)

Classe `String` (1 de 2)

- Uma *string* pode conter qualquer número de caracteres, inclusive, zero caracteres (denominada por *string* vazia: `s3`)
 - Ex: `String s1 = "Java", s2 = new String("Java"), s3 = "";`
- Uma *string* em Java é imutável (não permite a alteração de qualquer caractere da sequência)
- Um literal do tipo `String` é expresso por uma sequência de caracteres delimitada pelos caracteres aspas
 - As aspas não fazem parte da sequência de caracteres da *string*
- Disponibiliza o método `int length()` que retorna o número de caracteres na *string* (não inclui o caractere terminador)
 - Ex: `int n = "1234567".length(); // Produz o valor 7`

Classe `String` (2 de 2)

- Disponibiliza o método `char charAt(int index)` que retorna o valor Unicode do caractere na posição `index`
 - O primeiro caractere da *string* ocupa o índice 0 e o último caractere ocupa o índice `length() - 1`
 - Ex:
`String s2 = "Javax";`
`char c1 = s2.charAt(0); // Produz o valor 'J'`
`char c2 = s2.charAt(s2.length() - 1); // Produz o valor 'x'`
- A concatenação de *strings* é realizada através do operador `+`
 - Suporta a concatenação de uma *string* com um valor de outro tipo qualquer
 - Ex: `String s3 = "Valor = " + 45; // Produz a string "Valor = 45"`

Caracteres de *escape*

- Como incluir numa *string* o caractere aspas?
 - Com o caractere de escape `\` seguido de aspas, ou seja, o caractere `\"`
 - Embora escrito com dois caracteres, a sequência de *escape* é interpretada como um único caractere

- Ex:

```
String s = "\"Java\"";  
char c = s.charAt(0); // Retorna o caractere '"'  
int n = s.length(); // Retorna o valor 6
```

- Aplica-se o mesmo para a definição de literais do tipo caractere

Sequências de *escape*

Sequência de <i>escape</i>	Valor Unicode expresso em hexadecimal
\b	\u0008: <i>backspace BS</i>
\t	\u0009: <i>horizontal tab HT</i>
\n	\u000a: <i>linefeed LF</i>
\f	\u000c: <i>form feed FF</i>
\r	\u000d: <i>carriage return CR</i>
\”	\u0022: <i>double quote “</i>
\’	\u0027: <i>single quote ‘</i>
\\	\u005c: <i>backslash \</i>

Comparação de objetos `String` (1 de 2)

- O operador `==` para comparar duas *strings* apenas avalia se referenciam o mesmo objeto
- Disponibiliza o método `equals(Object other)` para avaliar se duas *strings* contêm a mesma sequência de caracteres
 - Ex:

```
String s1 = "Java";  
boolean b1 = s1 == new String("Java"); // Produz o valor false  
boolean b2 = (new String("Java")).equals(s1); // Produz o valor true
```
- Disponibiliza o método `equalsIgnoreCase(String other)` para avaliar a igualdade não considerando maiúsculas ou minúsculas
 - Ex:

```
boolean b3 = "Java".equals("java"); // Produz o valor false  
boolean b4 = "Java".equalsIgnoreCase("java"); // Produz o valor true
```

Comparação de objetos `String` (2 de 2)

- Disponibiliza o método `s.compareTo(String other)` que retorna um inteiro que compara a ordem lexicográfica de duas *strings*
 - < 0 , se `s` menor que `other`
 - $= 0$, se `s` igual a `other`
 - > 0 , se `s` maior que `other`
- A ordem lexicográfica é baseada nos valores dos caracteres em código Unicode
 - Os dígitos estão antes das letras
 - O alfabeto das letras maiúsculas está antes do alfabeto das letras minúsculas
- Disponibiliza os métodos `toUpperCase()` e `toLowerCase()` que retornam, respetivamente, uma nova *string* apenas com caracteres maiúsculos e uma nova *string* apenas com caracteres minúsculos

Resumo

- Visão geral do Java (cp. 1)
- Tipos primitivos (cp. 2)
- *Strings* (cp. 2)
- Escrita e leitura da consola (cp. 2)
- Controlo de fluxo (cp. 3 e 4)

Escrita no *standard output*

```
System.out.println("Texto escrito na consola");
```

- `System` é uma classe incluída no *package* `java.lang`
- `out` é um campo estático da classe `System` que guarda uma referência para um objeto do tipo `PrintStream` (neste caso representa o *standard output*)
- `println` é um método da classe `PrintStream`
 - Sobrecarga deste método para os diferentes tipos primitivos, `String` e `Object`
- Outros métodos
 - `println()` sem parâmetros escreve uma mudança de linha
 - `print(<type> param)` igual ao método `println` mas sem a mudança de linha no final do texto
 - `printf(String format, Object ... args)` equivalente ao `printf` do C

Leitura do *standard input*

```
Scanner keyboard = new Scanner(System.in);  
int v = keyboard.nextInt();
```

- `in` é um campo estático da classe `System` que guarda uma referência para um objeto do tipo `InputStream` (neste caso representa o *standard input*)
- `new` é o operador Java usado para criar instâncias de classes (objetos)
- `Scanner` é uma classe incluída no *package* `java.util` usada para ler valores de diferentes tipos do *stream* de entrada que é especificado na criação do `Scanner`
- `nextInt` é um método da classe `Scanner` para ler inteiros

Outros métodos da classe Scanner

- `nextByte()` devolve um valor do tipo `byte`
- `nextLong()` devolve um valor do tipo `long`
- `nextDouble()` devolve um valor do tipo `double`
- `next()` devolve um valor do tipo `String` correspondente a uma palavra
- `nextLine()` devolve uma `String` com todos os caracteres até ao final da linha (não inclui o caractere mudança de linha)
- `hasNextInt()` devolve se o próximo *token* pode ser interpretado como um inteiro
- `hasNext()` devolve se está disponível um próximo *token* (palavra)
- `hasNextLine()` devolve se está disponível uma linha

Método `nextLine`

- Por vezes pode ser mal usado quando misturado com outros métodos da classe `Scanner` (`nextByte`, `nextInt`, `next`)

- Ex:

```
Scanner keyboard = new Scanner(System.in);  
double d = keyboard.nextDouble();  
String tk = keyboard.next();  
String line1 = keyboard.nextLine();  
String line2 = keyboard.nextLine();
```

```
> 10,7 word  
> Linha de texto  
>
```

- Para o exemplo de *input* ao lado, os valores das variáveis são
 - > `d=10.7`
 - > `tk = "word"`
 - > `line1 = ""`
 - > `line2 = "Linha de texto"`

Resumo

- Visão geral do Java (cp. 1)
- Tipos primitivos (cp. 2)
- *Strings* (cp. 2)
- Escrita e leitura da consola (cp. 2)
- Controlo de fluxo (cp. 3 e 4)

Instruções de controlo de fluxo

- Decisão binária
 - Escolhe entre duas ações possíveis: instrução `if-else` e operador ternário
- Decisão múltipla
 - Permite executar um bloco de instruções entre várias alternativas: instrução `switch`
- Repetição
 - Permite executar um bloco de instruções repetidamente até ocorrer uma condição de paragem: instruções `while`, `for`, `do-while` e `for-each`

Instrução `if-else`

- Sintaxe

```
if (<expressão_booleana>) <instrução>  
[else <instrução>]
```

- O bloco `else` é opcional
- Se o bloco `else` for omitido e a expressão-booleana for `false`, nenhuma ação é executada
- A <instrução> se corresponder a um bloco de instruções deve ser envolvida entre os caracteres chavetas: { <instruções> }

Instrução `switch`

- Sintaxe

```
switch (<expressão_inteira>) {  
    case <expressão>[, ...]: <instrução>; break;  
    case <expressão>[, ...]: <instrução>; break;  
    ...  
    [default: <instrução>; break;]  
}
```

- Pode conter um mais casos
- Cada caso pode conter uma ou mais expressões separadas pelo caractere ‘,’ e terminado pelo caractere ‘:’
- O bloco de instruções de cada caso não precisa estar envolvido entre os caracteres chavetas
- Cada caso termina com a instrução `break`
- A cláusula `default` é opcional e não tem de ser a última

Instrução **switch** que produz um valor ($\geq$ Java 12)

- Sintaxe

```
<type> v = switch (<expressão_inteira>) {  
    case <expressão1_0>[, <expressão1_1>, ...] -> <expressão>;  
    case <expressão2_0>[, <expressão2_1>, ...] -> {  
        <instrução>*  
        yield <expressão>;  
    }  
    default -> <expressão>; | {<instrução>;* yield  
        <expressão>; }  
}
```

- Sintaxe semelhante à instrução clássica **switch** com as seguintes diferenças:

- Usado o separador seta (->) para separar casos do bloco a executar
- Não existe a instrução **break**
- Casos com múltiplas instruções são envolvidos com os caracteres chavetas e têm de terminar com a instrução **yield** seguida da expressão a retornar
- A cláusula **default** não é opcional e tem de ser a última

Instruções de repetição

- Sintaxe

```
while (<expr_controlo>) <instrução>
for ([expr_init]; [expr_controlo]; [expr_updt])
    <instrução>
do <instrução> while (<expr_controlo>);
for (<type> id: <coleção_iterável>) <instrução>
```

- A <instrução> se corresponder a um bloco de instruções deve ser envolvida entre os caracteres chavetas: { <instruções> }
- Na instrução for, as três expressões são opcionais, inclusive as três em simultâneo: denomina-se por *for forever* e é equivalente a um `while(true): for(;;){...}`
- A instrução *for-each* realiza as instruções do bloco do ciclo para cada elemento da coleção sendo que, em cada iteração, a variável `id` assume um valor sequencial da coleção (começa com o primeiro elemento da coleção)
- A instrução `break` quebra a instrução de repetição (evitar)
- A instrução `continue` transita de imediato a execução do programa para a próxima iteração do ciclo (evitar)